

Atelier 2 : enregistrer et recharger le registre

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Chapitre** : 2, Les fichiers · **Version** : 2026-10-23

Avant de commencer

Ce que vous allez construire	Le programme qui charge le registre des livraisons d'octobre de Yonnee Express, refuse les lignes invalides en expliquant pourquoi, écrit le fichier des rejets, et réenregistre le registre propre en CSV et en JSON
Durée	🟢 Débutant : 4 h · 🔵 Standard : 3 h · 🟣 Expert : 2 h (sans les défis)
Prérequis	Avoir lu le chapitre 2 ; votre dépôt `yonnee-express` de l'atelier 1
Fichiers fournis	Le registre d'octobre (`registre_octobre.csv` , 60 livraisons)
Rendu	Votre dépôt GitHub `yonnee-express` , avec l'étiquette `atelier-2`

Le registre a été saisi à la main par les livreurs. Il contient **des erreurs** : votre programme doit les trouver toutes, sans jamais planter. Ses colonnes :

Colonne	Contenu	Règle
`numero`	Le numéro de la livraison	Entier
`date`	La date, au format `AAAA-MM-JJ`	Une vraie date
`livreur`	Le prénom du livreur	Non vide
`quartier`	Le quartier de livraison	Un quartier desservi (tarifs de l'atelier 1)
`poids`	Le poids du colis, en kg	Nombre positif, avec un **point** décimal
`montant`	La valeur du colis, en F	De 500 à 500 000
`frais`	Les frais facturés, en F	Égaux au tarif du quartier, + 500 F au-delà de 5 kg
`duree_min`	La durée de la livraison, en minutes	Entier

Ce que votre programme affichera à la fin :

```
donnees\tarifs.json introuvable : tarifs par défaut.
53 livraisons valides, 7 lignes rejetées :
  ligne 6 : could not convert string to float: '2,5'
  ligne 13 : quartier inconnu : Ouakam
  ligne 19 : day is out of range for month
  ligne 25 : montant hors limites : -3000
  ligne 32 : frais incohérents : 999 au lieu de 1000
  ligne 40 : livreur vide
  ligne 47 : could not convert string to float: ''
Total des frais : 109500 F
Relecture du JSON identique : True
Fichiers écrits : registre.json, registre_propre.csv, rejets.csv
```

> 📍 **Point de départ** : votre dépôt `yonnee-express` de l'atelier 1. Vous y ajoutez un nouveau fichier `registre.py` ; `livraisons.py` (l'atelier 1) ne change pas.

Comment travailler

Comme à l'atelier 1 : chaque étape suit le plan 🎯 Objectif / 🛠️ A faire / 🔎 A chercher / 💡 A comprendre / ✅ Vérification ; une étape réussie = un commit. Lancez toujours le programme **depuis le dossier du dépôt** (`python registre.py`), sinon les chemins relatifs ne marcheront pas (chapitre 2, section 4).

Étape 0 : préparer les dossiers (10 min)

🎯 **Objectif** : un dossier `donnees` pour les fichiers d'entrée, un dossier `sorties` pour ce que le programme produit.

🛠️ **A faire**

1. Dans le dépôt, créez le dossier `donnees` et placez-y le fichier fourni `registre_octobre.csv` .
2. Créez `registre.py` avec les constantes de l'atelier 1 (`TARIFS_PAR_DEFAUT` , `SUPPLEMENT_LOURD` , `POIDS_LOURD` , `MONTANT_MIN` , `MONTANT_MAX`) et la liste `COLONNES` des 8 colonnes.
3. Ajoutez au fichier `.gitignore` une ligne `sorties/` : les fichiers produits ne se mettent pas dans Git (on peut toujours les refaire).

💡 **A comprendre** : pourquoi met-on le fichier d'entrée dans Git, mais pas les fichiers de sortie ?

✅ **Vérification** : `git status` montre `registre.py` et `donnees/registre_octobre.csv` , pas de dossier `sorties` .

Étape 1 : les tarifs dans un fichier de configuration (25 min)

🎯 **Objectif** : changer un tarif sans toucher au code.

🛠️ **A faire** : écrivez `charger_tarifs(chemin)`, qui lit un dictionnaire de tarifs dans un fichier JSON et renvoie **une copie** des tarifs par défaut (`dict(TARIFS_PAR_DEFAUT)`) si le fichier est absent ou abîmé, avec un message dans chaque cas.

🔎 **A chercher** : chapitre 2, sections 7 et 8.

✅ **Vérification** : sans fichier : « ... introuvable : tarifs par défaut. » ; avec un fichier `donnees/tarifs.json` contenant `{"Médina": 1000, "Yoff": }` : « ... abîmé (Expecting value: ...) : tarifs par défaut. » ; avec un fichier correct : ses tarifs. Supprimez ensuite le fichier de test.

```
git add .
git commit -m "Tarifs lus dans un fichier JSON"
```

Étape 2 : convertir et valider une ligne (45 min)

🎯 **Objectif** : une fonction qui transforme une ligne du CSV en vraie livraison, ou lève une `ValueError` qui dit **pourquoi**.

🛠️ **A faire** : écrivez `frais_livraison(tarifs, quartier, poids)` (comme à l'atelier 1), puis `convertir(ligne, tarifs)`, qui vérifie dans cet ordre :

1. aucune colonne manquante (`None in ligne.values()`) : sinon `ValueError("colonne manquante")` ;
2. la date, avec `date.fromisoformat(...)` (du module `datetime`) ;
3. le livreur, non vide après `strip()` ;
4. le quartier, connu des tarifs ;
5. le poids, converti avec `float`, et positif ;
6. le montant, converti avec `int`, entre les bornes ;
7. les frais, convertis avec `int`, et **égaux** à `frais_livraison(...)`.

Elle renvoie un dictionnaire avec les bons types : `numero` et `duree_min` en `int`, `date` en objet `date`, `poids` en `float`...

▼ 💡 Indice : le début de la fonction

```python
def convertir(ligne, tarifs):
    if None in ligne.values():
        raise ValueError("colonne manquante")
    jour = date.fromisoformat(ligne["date"])
    livreur = ligne["livreur"].strip()
    if not livreur:
        raise ValueError("livreur vide")
    # ... la suite
```

`date.fromisoformat`, `float` et `int` lèvent déjà une `ValueError` quand le texte est invalide : inutile de les entourer d'un `try` ici, l'exception remontera (chapitre 1, section 5.2).

💡 **A comprendre**

- Pourquoi vérifier la colonne manquante **en premier** ? Que se passerait-il sinon avec `ligne["livreur"].strip()` ?
- Pourquoi vérifier les frais : le livreur ne pouvait-il pas se contenter de les recopier ?

✅ **Vérification** : dans le terminal Python (`python` , puis `from registre import *`), `convertir({"numero": "1", "date": "2026-10-04", "livreur": "Modou", "quartier": "Médina", "poids": "4.0", "montant": "143000", "frais": "1000", "duree_min": "48"}, TARIFS_PAR_DEFAUT)` renvoie un dictionnaire dont la date est `datetime.date(2026, 10, 4)` . Changez `"frais"` en `"1500"` : `ValueError: frais incohérents : 1500 au lieu de 1000` .

```
git add .
git commit -m "Conversion et validation d'une ligne"
```

Étape 3 : charger le registre et signaler les rejets (35 min)

🎯 **Objectif** : lire tout le fichier sans planter.

🛠️ **A faire**

1. `charger_registre(chemin, tarifs)` renvoie `(valides, rejets)` ; chaque rejet est un tuple (numéro de ligne dans le fichier, raison, contenu brut de la ligne). Le contenu brut : les valeurs de la ligne recollées avec `";".join(str(v) for v in ligne.values())` .
2. Une fonction `main()` qui charge le registre et affiche le bilan (comme au début de l'énoncé) et le total des frais des livraisons valides. Si le fichier du registre est introuvable, un message clair, sans traceback.
3. En bas du fichier : `if __name__ == "__main__": main()` (comme à l'atelier 1).

✅ **Vérification** : **53** livraisons valides, **7** rejets (lignes 6, 13, 19, 25, 32, 40, 47), total des frais **109 500 F**. Renommez un instant le fichier du registre : le message « Le registre est introuvable... » s'affiche.

💡 **A comprendre**

- La ligne 19 contient la date `2026-10-32` . Quel est le message, et d'où vient-il ?
- La ligne 6 contient `2,5` . Faut-il la refuser ou la corriger ? Relisez l'encadré 🔵 de la section 6 du chapitre et donnez votre avis.

```
git add .
git commit -m "Chargement du registre et bilan des rejets"
```

Étape 4 : écrire les rejets et le registre propre (30 min)

🎯 **Objectif** : laisser une trace que l'équipe pourra corriger, et un registre propre.

🛠️ **A faire**

1. Le dossier `sorties` est créé par le programme s'il n'existe pas (`Path(...).mkdir(exist_ok=True)`).
2. `ecrire_rejets(rejets, chemin)` écrit `sorties/rejets.csv` , avec l'en-tête `ligne;raison;contenu` .
3. `ecrire_registre(livraisons, chemin)` écrit `sorties/registre_propre.csv` , avec les 8 colonnes ; la date doit être reconvertie en texte.

<details>
<summary>💡 Indice : convertir la date au moment d'écrire</summary>

`{**livraison, "date": livraison["date"].isoformat()}` crée une **copie** du dictionnaire où seule la date est remplacée par son texte. La livraison d'origine n'est pas modifiée.
</details>

✅ **Vérification** : `sorties/rejets.csv` a 8 lignes (l'en-tête et 7 rejets) et s'ouvre correctement dans Excel ; `sorties/registre_propre.csv` a 54 lignes.

💡 **A comprendre** : dans `rejets.csv` , la colonne `contenu` est entourée de guillemets (`"54;2026-10-01;Coumba;..."`). Pourquoi le module `csv` les a-t-il ajoutés ?

```
git add .
git commit -m "Fichier des rejets et registre propre"
```

Étape 5 : le registre en JSON, aller et retour (30 min)

🎯 **Objectif** : enregistrer le registre avec ses types, et vérifier qu'on le relit à l'identique.

🛠️ **A faire**

1. `sauver_json(livraisons, chemin)` : une liste de dictionnaires, dates en texte, `indent=2` , `ensure_ascii=False` .
2. `charger_json(chemin)` : relit le fichier et reconvertit chaque date avec `date.fromisoformat` .
3. Dans `main()` , après l'écriture : relisez le fichier et affichez `Relecture du JSON identique : ...` (la comparaison `relu == valides`), puis la liste triée des fichiers du dossier `sorties` .

✅ **Vérification** : `Relecture du JSON identique : True` ; `Fichiers écrits : registre.json, registre_propre.csv, rejets.csv` .

💡 **A comprendre** : comparez la taille des deux fichiers (`registre.json` et `registre_propre.csv` , clic droit dans VS Code ou `Path(...).stat().st_size`). Lequel est le plus gros, et pourquoi ? Quand préférer l'un ou l'autre ?

```
git add .
git commit -m "Registre en JSON"
```

Étape 6 : un nouveau quartier, sans toucher au code (15 min)

🎯 **Objectif** : vérifier que la configuration marche vraiment.

🛠️ **A faire** : Yonnee Express dessert désormais **Ouakam**, à 2 000 F. Créez `donnees/tarifs.json` avec les six quartiers, et relancez.

✅ **Vérification** : la ligne 13 n'est plus rejetée pour « quartier inconnu », mais pour `frais incohérents : 1500 au lieu de 2000` : le livreur avait appliqué un mauvais tarif. Il reste 7 rejets.

💡 **A comprendre** : faut-il mettre `donnees/tarifs.json` dans Git ? Pourquoi ?

Puis étiquetez :

```
git add .
git commit -m "Ouakam ajouté par la configuration"
git tag atelier-2
git push
git push origin atelier-2
```

🏆 Défis (facultatifs)

1. **Corriger ce qui est sûr.** Acceptez la virgule décimale dans le poids (en la remplaçant par un point) et comptez à part les lignes **corrigées**. Relancez : la ligne 6 est-elle maintenant acceptée ? Que vous apprend la nouvelle raison du rejet sur le danger de corriger un fichier en silence ?
2. **Le rapport texte.** Écrivez `sorties/rapport.txt`, lisible par un humain : la date et l'heure de l'exécution, le nombre de lignes lues, valides et rejetées, le total des frais, et la liste des rejets.
3. **Les archives.** A chaque exécution, copiez le registre propre dans `sorties/archives/registre_AAAA-MM-JJ_HHMM.csv` (avec `datetime.now().strftime(...)`), sans jamais écraser une archive existante (mode `"x"`).
4. **Un registre en cp1252.** Enregistrez le fichier fourni avec l'encodage cp1252 (ouvrez-le en UTF-8, réécrivez-le en cp1252) et faites en sorte que votre programme lise les deux versions : il essaie UTF-8, et passe à cp1252 en cas de `UnicodeDecodeError`.

Grille d'autoévaluation

Je sais…	Oui	Pas encore
lire et écrire un fichier CSV avec `DictReader` et `DictWriter`	☐	☐
valider une ligne et lever une `ValueError` qui dit pourquoi	☐	☐
lire un fichier abîmé sans planter, en gardant le numéro de ligne des rejets	☐	☐
enregistrer et relire des données en JSON, dates comprises	☐	☐

Je sais…	Oui	Pas encore
lire une configuration avec des valeurs par défaut	☐	☐
organiser les fichiers d'un projet avec `pathlib`	☐	☐